

IA Aplicada com LLMs

Aula 08 — Escrever e ler

Parte 1

Três políticas de escrita

As três, comparadas

Política	Custo	Cobertura	Modo de falha
O agente decide	uma decisão por passo	captura o imprevisto	armazena ruído; a memória degrada
O código extrai	uma chamada por execução	só o previsto no prompt	perde o que a extração não previu
O humano corrige	trabalho humano	a mais confiável	não escala

Nenhuma é gratuita, e a escolha é de projeto — não de preferência.

O agente decide

```
def lembrar(contenido: str, tipo: str) -> str:  
    """Grava um fato na memória de longo prazo."""
```

A vantagem é real: o agente tem acesso ao **raciocínio** que produziu o fato, e sabe o que foi surpreendente.

O modo de falha decorre da mesma propriedade que torna a Aula 05 necessária:

Um agente sem restrição explícita tende a usar a ferramenta que tem.

E a restrição adequada é **arquitetural, não instrucional**: um teto de gravações por execução, imposto em código.

O código extrai

```
def extrair_pelo_codigo(execucao: str) -> list[dict]:  
    bruto = _chamar(PROMPT_EXTRACAO.format(execucao=execucao),  
                    schema=SCHEMA_EXTRACAO)  
    return json.loads(bruto)["fatos"]
```

Custo previsível — uma chamada por execução, independentemente do número de passos — e comportamento uniforme.

É a política adequada quando o domínio é conhecido e o que interessa guardar é enumerável.

O prompt de extração é prompt de produção

```
PRESERVE OBRIGATORIAMENTE como `episodica` toda AÇÃO DE ESCRITA executada, com o identificador que ela devolveu (ex: "registrou o parecer P-1188 para a despesa D-4612").
```

Perder essa informação faz o agente **repetir a escrita** numa execução futura.

É a **mesma exigência** do prompt de compaction da Aula 05 — aqui **entre** execuções, em vez de dentro de uma.

As duas listas de "preservar obrigatoriamente"

Aula 05 — compaction	Aula 08 — extração
objetivo	— (a próxima execução tem outro)
decisões e razões	decisões e razões
escritas com id	escritas com id
erros cometidos	erros cometidos
fatos numéricos	fatos sobre entidades

Duas listas quase iguais, produzidas por problemas diferentes — o que é uma boa indicação de que a lista está certa.

Parte 2

Escrever é escrita

Gravar memória tem efeito no mundo

```
def gravar(self, fato: dict, chave: str) -> dict:
    if (existente := MEMORIA.get(chave)):
        return {**existente, "ja_existia": True}
    ...
```

A **chave de idempotência** da Aula 05, aplicada a um lugar em que quase ninguém a aplica.

```
1ª gravação -> M-0042 ja_existia=false
2ª gravação -> M-0042 ja_existia=true
```

`uuid4()` a cada chamada **não** é chave de idempotência — é um identificador novo por tentativa.

A procedural exige idempotência de outra natureza

Uma regra episódica repetida ocupa espaço. Uma **regra procedural** repetida entra no *system prompt* duas vezes — e o modelo passa a tratá-la como mais importante.

```
system: "...quando o recibo diverge, devolver antes de analisar.  
        ...quando o recibo diverge, devolver antes de analisar.  
        ...quando o recibo diverge, devolver antes de analisar."
```

A chave, aqui, precisa ser derivada do **conteúdo semântico**, e não do texto literal — duas redações da mesma regra são a mesma regra.

Parte 3

A leitura

Memória compete por janela

```
@dataclass
class Orcamento:
    max_tokens_memoria: int = 800

    def recuperar(self, consulta, k=3) -> dict:
        ...
        return {"texto": texto, "tokens": tokens}
```

O método devolve o custo junto com o conteúdo.

Sem o número de tokens, não há como afirmar que a memória cabe — e memória que não cabe empurra o objetivo do usuário para a região do meio, que o modelo lê pior (Aula 05, nota 03).

A consulta que combina os três mecanismos

pergunta: "a despesa D-5510 do F-088 em Lisboa está dentro da política?"

semântica	F-088 -> "viaja para Portugal com frequência"	0 chamadas
episódica	busca "Lisboa hospedagem" + filtro funcionario -> 2 episódios, ambos aprovados pelo art. 19	1 embedding
procedural	já está no system prompt	0 chamadas

total: 1 chamada de embedding, ~80 tokens de contexto

Três mecanismos, um custo, e nenhum deles é o índice inteiro.

Parte 4

A trajetória também compete

A memória não é a única que enche a janela

A §4 orçou a **memória**. Falta a fonte que cresce mais rápido:
a **trajetória da execução corrente**.

A Aula 05 fez a aritmética — o acumulado cresce com o **quadrado** dos passos.
As técnicas ficaram para cá por um motivo: **comprimir e memorizar são a mesma decisão**.

O que enche a janela

Fonte	Volume	Ainda é útil?
Resultados de ferramenta	o maior, de longe	quase sempre não
Raciocínio intermediário	médio	raramente
Declarações de ferramentas	fixo, em toda volta	enquanto ativas
System + objetivo	pequeno	sempre

O alvo **não** é o diálogo. São os resultados de ferramenta.

Tool clearing — a mais barata

```
{"role": "tool",  
  "content": "[resultado removido] consultar_politica(categoria=refeicao)  
              -> teto R$ 120,00 por refeição; exige nota fiscal"}
```

Seletiva (só o que já foi consumido) · **reversível** (o dado completo continua no estado) · **preserva a estrutura** da trajetória · **é código, não modelo** (zero chamadas extras).

Apagar a chamada inteira faz o agente repeti-la.

Sem registro da consulta anterior, ele consulta novamente — e a otimização produziu um laço.

Compaction: reativa × periódica

	Reativa	Periódica
Dispara	ao cruzar limiar	a cada N passos
Custo	menor	maior
Previsibilidade	baixa	alta
Risco	1 resultado gigante estoura antes do teste	comprimir cedo demais

Na prática: **periódica como piso, reativa como teto.**

Passos	Sem compaction	Com (a cada 10)
20	121.200	85.200
30	253.800	133.800

Sumarização — o que NUNCA pode sair

Preservar	Por quê
Objetivo	é a âncora — sem ele, deriva
Decisões tomadas	ou o agente reconclui
Escritas executadas + ids	ou registra duas vezes
Erros cometidos	ou repete a tentativa
Fatos numéricos	a decisão final vai citá-los

Pode ir: raciocínio intermediário, resultados consumidos, becos sem saída.

O prompt de compaction é um prompt de produção

```
PROMPT_COMPACTACAO = """Resuma a trajetória para que outro agente continue do ponto em que ela parou.
```

```
PRESERVE OBRIGATORIAMENTE: decisões e suas razões; ações de ESCRITA executadas com os identificadores; erros cometidos; fatos numéricos.
```

```
DESCARTE: raciocínio intermediário, resultados já usados.
```

```
Não conclua a tarefa. Não invente informação."""
```

Contrato de saída, exemplos negativos, formato — tudo da Aula 03.

Versionado e testado como qualquer outro.

E o preço: perda de informação com aparência de continuidade

O agente segue funcionando, com uma memória que alguém editou.

Quando ele errar por isso, não vai parecer erro de memória.

Vai parecer burrice do modelo.

Uma compaction que quebrou o agente

```
resumo = """O agente analisou a despesa D-4471, consultou a política  
e concluiu que está dentro do teto. O parecer foi elaborado."""
```

Sumiu: o id do parecer registrado (P-1001).

Passo seguinte: o agente chamou registrar_parecer de novo.

O que salvou: a chave de idempotência.

As salvaguardas se cobrem umas às outras. Nenhuma basta sozinha —
é por isso que a arquitetura tem camadas, e não um remédio.

O que fica

- Três políticas de escrita, e **nenhuma é gratuita**
- A restrição ao agente é **arquitetural**, não instrucional
- **Gravar memória é escrita** — chave de idempotência, e `ja_existia` no retorno
- Memória **compete por janela**, e o custo volta junto com o conteúdo
- **Comprimir trajetória e escrever memória são a mesma decisão** — as duas listas de "preservar obrigatoriamente" são quase idênticas

Próxima nota: o que acontece quando dois fatos verdadeiros **se contradizem** — e como se apaga o que não deve mais existir.